

Concordia University - Department of Electrical and Computer Engineering
Applied Machine Learning - COEN 330
Summer 2026

Course Project Guidelines

Extended Version of the Project Overview

Instructor: Dr. Ahmed Bali

Document purpose: This document extends the Project Overview previously shared with students. It provides the detailed project requirements, quality expectations, deliverables, and evaluation criteria. The project must demonstrate an end-to-end machine learning workflow: problem definition, data preparation, model development, validation, evaluation, interpretation, and reproducible reporting.

1. Project Overview

The course project is a core component of Applied Machine Learning. It gives students the opportunity to apply machine learning techniques to a real-world or realistic engineering problem from start to finish. The project emphasizes the complete applied ML workflow: problem formulation, data understanding, preprocessing, feature engineering, model development, validation, evaluation, interpretation, reproducibility, and reporting.

Students are expected to select a problem that is meaningful, feasible within the accelerated session, and rich enough to demonstrate several concepts covered in class.

2. Project Objectives

- Formulate an engineering, computer science, or data-driven application problem as a machine learning task.
- Prepare and analyze a dataset using preprocessing, feature engineering, and exploratory data analysis.
- Implement and compare several machine learning models, including a clear baseline.
- Use validation and hyperparameter tuning to support model selection.
- Evaluate models using appropriate task-specific metrics.
- Interpret results beyond raw numerical scores through error analysis and qualitative discussion.
- Organize code, notebooks, results, and documentation so that the work is understandable and reproducible.
- Communicate the project professionally through a final report and, if requested, a short presentation or demo.

3. General Project Idea and Accepted Problem Types

Each team must choose a machine learning problem related to engineering, computer science, or another relevant applied domain. The topic must be sufficiently scoped for a short accelerated session and must allow meaningful model development and evaluation.

Project category	Example topics
Classification	Fault detection, intrusion detection, disease prediction, defect classification, spam detection.
Text classification	Sentiment analysis, topic classification, spam classification, support-ticket classification.
Image classification	Object classification, defect classification, basic computer-vision applications.
Regression	Energy consumption prediction, temperature prediction, performance estimation, traffic volume prediction.
Time series forecasting	Sensor prediction, energy demand forecasting, network traffic forecasting, workload forecasting.
Anomaly detection	Abnormal sensor behavior, cyberattack detection, equipment failure detection, fraud detection.
Clustering or dimensionality reduction	Customer/device grouping, sensor pattern discovery, PCA-based visualization or preprocessing.
Reinforcement learning	May be accepted only with instructor approval because it requires a different formulation and evaluation process.

4. Minimum Technical Requirements

4.1 Problem Definition

The final report must clearly define:

- the problem being addressed;
- the motivation and application context;
- the input features and the target/output variable;
- the type of ML task: classification, regression, clustering, forecasting, anomaly detection, etc.;
- the expected practical value of the solution;
- the assumptions and scope limitations of the project.

4.2 Dataset and Data Quality Requirements

The project must include a clear description of the dataset and all preprocessing decisions. Students must document:

- dataset source or collection method;
- number of samples and features;
- description of the target variable or labels, when applicable;
- data types: numerical, categorical, text, images, time series, etc.;
- handling of missing values and outliers, if applicable;
- scaling, normalization, categorical encoding, text representation, image preprocessing, or time-series transformations, if applicable;
- train/validation/test split or cross-validation strategy;
- possible data leakage risks and how they were avoided;
- ethical, privacy, licensing, or usage constraints related to the dataset, if applicable.

Quality expectations for the dataset include:

- The dataset must be large and informative enough to support meaningful learning and evaluation.
- The dataset should not be a trivial dataset unless the team adds meaningful analysis, feature engineering, or comparison.
- The dataset should be connected to a practical domain application, preferably engineering, computer systems, networks, sensors, energy, software, cybersecurity, biomedical data, or another applied area.

4.3 Exploratory Data Analysis

The project must include exploratory data analysis with appropriate figures and observations, such as:

- distributions of important features;
- class balance or target distribution;
- correlations or feature relationships;
- missing-value analysis;
- outlier analysis;
- representative examples or visualizations;
- initial observations about the dataset and expected modeling difficulty.

4.4 Feature Engineering and Data Representation

Students must show that they understand how the data representation affects the ML model. Depending on the project, this may include:

- creating new features from domain knowledge;
- using polynomial, interaction, lag, statistical, text, or image features;
- encoding categorical variables;
- normalizing or standardizing numerical variables;
- using text representations such as bag-of-words, TF-IDF, or embeddings;
- using dimensionality reduction such as PCA when appropriate;
- justifying why specific features were kept, removed, transformed, or created.

4.5 Model Development

Each team must implement and compare at least five machine learning models (or variants). One of these models must be simple and interpretable and must serve as a baseline. The selected models must be appropriate to the task.

Task	Possible models
Classification	Logistic regression, k-NN, SVM, decision tree, random forest, gradient boosting, Naive Bayes, neural network.
Regression	Linear regression, ridge/lasso regression, k-NN regression, decision tree regression, random forest regression, gradient boosting regression, neural network regression.
Clustering	k-means, hierarchical clustering, DBSCAN, Gaussian mixture model. When the task is unsupervised, model comparison may involve different algorithms, different numbers of clusters, or different feature representations.
Time series forecasting	Naive baseline forecasting, linear regression with lag features, random forest, gradient boosting, simple neural network, LSTM if appropriate.
Anomaly detection	Isolation forest, one-class SVM, local outlier factor, autoencoder, clustering-based anomaly detection.

Note: The model development section must include justification for selecting these models.

5.6 Validation, Model Selection, and Hyperparameter Tuning

Students must not select the final model (hyperparameters) using the test set. The test set must be reserved for final evaluation only. The project must include a clear validation strategy, such as a train/validation/test split, k-fold cross-validation, or time-based validation for time-series problems.

At least three of the implemented models must include hyperparameter tuning using grid search, randomized search, or another justified search strategy. Examples include:

- Logistic regression: regularization strength C and penalty type, if applicable.
- SVM: C, kernel type, gamma of RBF kernel, degree for polynomial kernels.
- k-NN: number of neighbors k, distance metric, weighting strategy.
- Decision tree: maximum depth, minimum samples per split, criterion.
- Random forest or gradient boosting: number of estimators, maximum depth, learning rate, subsampling parameters.
- Neural networks: number of hidden layers, number of neurons, learning rate, batch size, epochs, regularization.

5.7 Model Evaluation

Students must use metrics appropriate to the selected task and must justify why these metrics are suitable.

Task	Possible metrics
Classification	Accuracy, precision, recall, F1-score, ROC-AUC, confusion matrix.
Imbalanced classification	Precision, recall, F1-score, PR-AUC, class-specific metrics, confusion matrix.
Regression	MAE, MSE, RMSE, R-squared.
Clustering	Silhouette score, Davies-Bouldin index, cluster visualization, qualitative cluster interpretation.
Time series forecasting	MAE, RMSE, MAPE, time-based validation, comparison with naive baseline.
Anomaly detection	Precision, recall, F1-score, confusion matrix if labels are available; qualitative inspection if labels are not available.

Note: The model evaluation section must include:

- a clear comparison table of model results.
- discussion of model complexity, interpretability, and computational cost when relevant.

5.8 Qualitative Analysis and Interpretation

The project must go beyond raw metrics. Students should include:

- error analysis;
- examples of correct and incorrect predictions, when applicable;
- discussion of false positives and false negatives, when applicable;
- interpretation of important features;
- comparison of model behavior and explanation of why some models performed better than others;
- limitations of the dataset and models;

- possible sources of error and future improvements.

5.9 Reproducibility

The submitted project must be reproducible. The submission must include:

- a clear folder structure;
- a README file with instructions to run the project;
- requirements.txt, environment.yml, or equivalent dependency information;
- notebooks or scripts for preprocessing, training, validation, and evaluation;
- dataset link or instructions to obtain the dataset;
- saved models, results, and figures if appropriate;

Note: it is important to use a clear random seed or explain the randomness where relevant.

5.10 Demo or Deployment Component

A minimal usage demonstration is expected as part of the reproducibility package. This does not need to be a polished production deployment. It should simply illustrate how the trained model can be used on new input or sample data. A more advanced deployment or user interface is optional and may receive bonus consideration.

Acceptable options include:

- an interactive Jupyter or Colab notebook;
- a command-line script that loads the model and predicts on sample input;
- a Streamlit or Gradio app;
- a Flask or FastAPI REST API;
- a simple local web interface;

Notes:

- *It is recommended that screenshots are taken or a short video is made demonstrating the model usage.*
- *The demo should help illustrate practical use of the model, but it should not take priority over sound ML methodology, validation, and evaluation.*

6. Submission Requirements

All project components must be submitted on Moodle. Since this is an accelerated course, there will be one official final submission deadline. No intermediate project deliverables are required unless announced separately by the instructor.

Each team must submit one compressed file named using the following format:

TeamName_Project.zip

The compressed file must include:

- Final report in PDF format.
- Source code and notebooks used for preprocessing, training, validation, evaluation, and visualization.
- README file with clear instructions explaining how to run the code and reproduce the main results.
- Dataset information: dataset link, source, description, access instructions, and preparation instructions.
- Results and figures: tables, plots, confusion matrices, evaluation metrics, and qualitative examples.
- Demo files: demo application, screenshots, or short video (If they are large, put their link).

7. Final Report Structure

The final report should be written in a clear technical style. IEEE-style formatting is recommended. The suggested length is 8 to 10 pages, excluding references and appendices.

The report must include the following sections:

1. Abstract

2. Introduction and motivation
3. Related work or background
4. Dataset description
5. Preprocessing and exploratory data analysis
6. Methodology and models
7. Validation and hyperparameter tuning strategy
8. Experimental setup
9. Results and model comparison
10. Error analysis and qualitative discussion
11. Demo or usage demonstration description
12. Limitations and future work
13. Conclusion
14. Team contribution section explaining each member's contribution.
15. References

8. Recommended Folder Structure

Students may organize their repository differently, but the following structure represents a recommended example:

```
project-name/
├── README.md
├── requirements.txt
├── data/
│   ├── raw/ or data_link.txt
│   └── processed/
├── notebooks/
│   ├── 01_eda.ipynb
│   ├── 02_preprocessing.ipynb
│   ├── 03_model_training.ipynb
│   └── 04_evaluation.ipynb
├── src/
│   ├── preprocessing.py
│   ├── train.py
│   ├── evaluate.py
│   └── utils.py
├── models/
│   └── saved_model.pkl
├── results/
│   ├── metrics_table.csv
│   ├── confusion_matrix.png
│   └── plots/
├── demo/
│   ├── app.py or demo_notebook.ipynb
│   └── README.md
└── report/
    └── final_report.pdf
```

9. Grading Rubric

The project will be graded according to the following criteria. If the instructor does not request a presentation, the presentation weight will be redistributed to the final report quality.

Component	Weight
Problem definition and motivation	10%
Dataset preparation, preprocessing, feature engineering, and exploratory data analysis (EDA)	15%
Model development and validation, including hyperparameter tuning of at least five candidate models	20%
Model evaluation and comparison of the tuned models	20%
Error analysis and interpretation of the selected model	5%
Reproducibility and minimal usage demonstration	10%
Final report quality	10%
Presentation and Q&A, if requested by the instructor	10%
Total	100%

10. Detailed Assessment Expectations

Criterion	Excellent performance
Problem formulation	The problem is clear, motivated, relevant, feasible, and well scoped as an ML task.
Data work	The dataset is well described, cleaned, analyzed, and prepared with justified preprocessing decisions including feature engineering or selection.
Modeling	At least five suitable models are implemented, including a baseline and stronger models.
Validation and tuning	A proper validation strategy is used, and key hyperparameters are tuned without using the test set for model selection.
Evaluation	Metrics are appropriate for the selected task and clearly justified.
Interpretation	The report includes meaningful error analysis, qualitative insights, and discussion of limitations.
Reproducibility	The code runs, the README is clear, and the environment is documented.
Demo	A minimal usage demonstration works and illustrates practical use of the model.
Report	The report is well written, well organized, and includes clear figures, tables, and explanations.
Presentation	The presentation is clear, concise, and technically sound, if requested.

Note: Additional clarifications and detailed requirements may be announced on Moodle. Students are responsible for checking Moodle regularly.

11. Bonus Features

Bonus consideration may be given, at the instructor's discretion, for high-quality additions such as:

- collecting an original dataset;
- using advanced interpretability methods such as SHAP or LIME;
- deploying the model online;
- implementing an advanced machine learning model appropriately, such as a neural network, deep learning model, reinforcement learning approach, or another advanced model suitable for the selected problem;
- adding a polished user interface;
- performing a meaningful fairness, bias, or robustness analysis;
- using experiment tracking tools such as MLflow or Weights & Biases;
- providing additional controlled experiments, such as comparing different feature sets, preprocessing strategies, model configurations, or validation settings.

12. Academic Integrity and Use of External Tools

All submitted work must comply with University academic integrity regulations. Students must not submit code, text, figures, or results that they did not produce or properly acknowledge.

Any external sources, datasets, code libraries, tutorials, or AI-assisted tools used in the project must be clearly acknowledged in the report and/or README file, according to the instructor's instructions. Students are responsible for understanding their submitted code and results. During a presentation or discussion, the instructor may ask students to explain any part of the submitted work.

13. Final Submission Checklist

☐ The problem is clearly formulated as an ML task.	☐ The dataset source and preparation steps are documented.
☐ EDA figures and observations are included.	☐ Preprocessing and feature engineering are explained.
☐ At least five models are implemented and compared.	☐ Hyperparameter tuning is included.
☐ Validation and test evaluation are separated.	☐ Metrics are appropriate and justified.
☐ Error analysis and qualitative interpretation are included.	☐ README and dependency files are provided.
☐ The code reproduces the main results (or similar results if randomness is involved), with seeds documented when applicable.	☐ A minimal usage demonstration is included.
☐ The final report is submitted as a PDF.	☐ The team contribution statement is included.